HEAPS

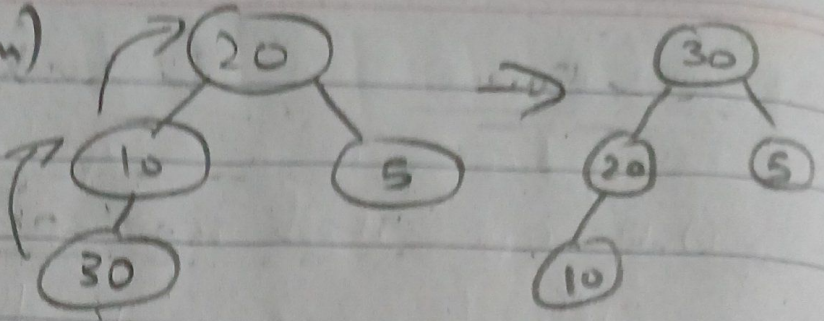
(.) Binary Heap is a complete Binary Tree
(.) It represented in an array no blank spaces in the array
(.) bcz then only these formulaes apply

Node at index i	0-based indexing	i	1-based indexing
left child	2i+1	2i	
Right Child	2i+2	2i+1	

(.) Heaps are also called almost complete Binary Tree.
(.) Can have duplicates unlike BST

(.) MAX HEAP every node has element greater than or equal to all of its descendants
(.) MINHEAP every node has ele less than or eq to all of its descendants.
(.) height of Heap (logn)
(.) Not useful for searching purposes.
(.) Heap is created in place, we don't need another array
(.) Heaps Retrival -> O(1)
(.) Heaps Insertion -> O(logn)
(.) Heap's Creation -> O(nlogn)
↳ with n elements.
(.) Heap's Deletion -> O(logn)

(•) Insertion (Swim)



O(log n)

Heap stored in 1-based Indexing in Array

```
(•)  void Insert ( int ray[], int n )
     {
        int temp = ray[n];
        int it = n;

        while( it>1 && temp > ray[it/2])
        {
            ray[it] = ray[it/2];
            it = it/2;
        }
        ray[it] = temp;
     }
```

Q) What is n in the parameters?

(•) It is the index where we first insert our element & then call the insert function for that 1 element which ensures that our heap structure stays intact.

(•) Creation — O (n log n)

random array ray

0	10	20	30	25	5	40	35
0	1	2	3	4	5	6	7

```
void createHeap (int ray[]);
{
    int ray[] = {0,10,20,30,25,5,40,35};
    int size = 7;

    for (int m=2; m<=7; m++)
    {
        insert (ray, m);
    }
}
```

(•) Deletion (Sink)

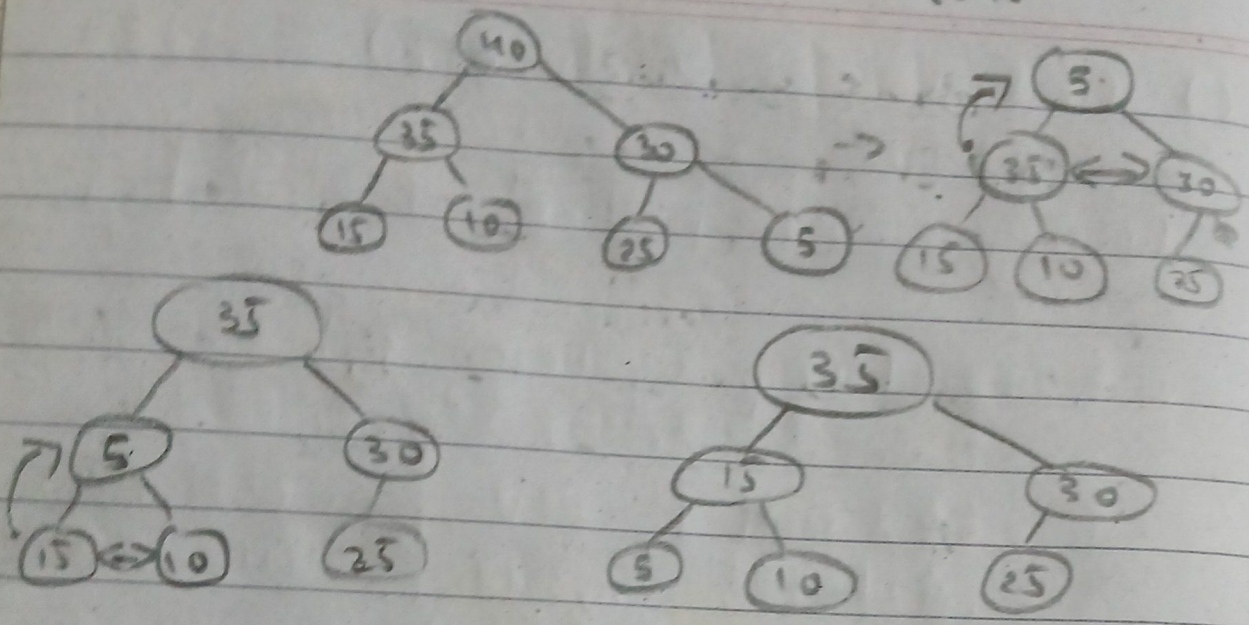
(•) We can only delete highest / lowest priority element / the root of heap.

(•) The one need to adjust ele after deletion in a way to ensure we have a complete binary tree (heap structure).

(•) The last leaf node takes the deleted ele's place then we perform sink.

	0	1	2	3	4	5	6	7
RAY	0	40	35	30	15	10	25	5

t = 40




```
void DeleteFromHeap ( int ray[], int n )
{
    int temp = ray[n];
    RAY[1] = RAY[n];
    int i = 1;          // element to adjust
    int j = 2*i;        // j is left child.

    while ( j <= n-1 )
    {
    if ( RAY[j+1] > RAY[j] )
    {
        j = j+1;
    }
    if ( RAY[i] < RAY[j] )
    {
        Swap ( RAY[i], RAY[j] );
    }
```

else { break; }
}
A[n] = temp;
}

(•) From Heap we go on del the ele one we del an ele we get a free space @ the end where we can store the del ele

(•) HeapSort O(n log n)

1) Create Heap of n elements.
2) Delete 'n' elements 1-by-1
   1) takes for n ele → O(n log n)
   2) takes for n ele → O(n log n)
   ≃ (n log n)

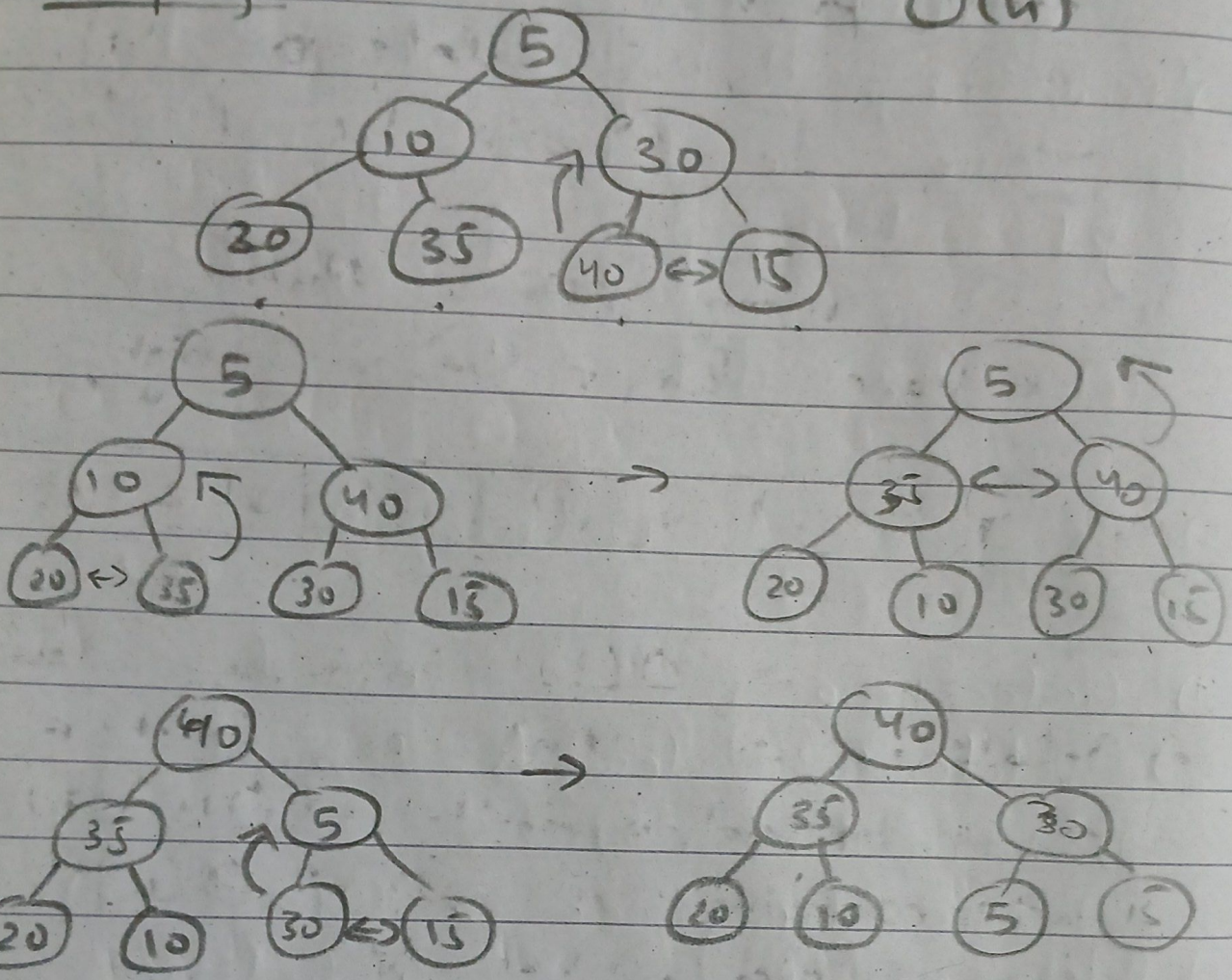
total time complexity O(2n log n)

HEAPIFY

(•) related to creation of a heap not just insertion of 1 element.
(•) Inserting all elements 1-by-1 in a heap & creating a heap
(•) faster method for creating a heap, as we don't process leafs

RAY | 5 | 10 | 30 | 20 | 35 | 40 | 15 |

Heapify $O(n)$



(·) We may not get the same Heap from both but we will get a Heap (valid)

PRIORITY QUEUE

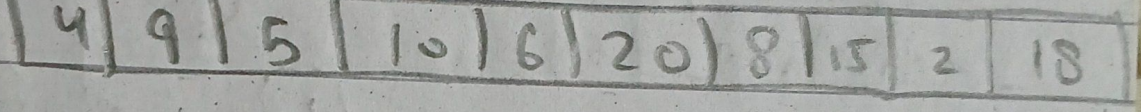
(·) related to insertion & deletion of elements

(·) elements have their priority

& they are inserted with their priority.

(+) Always during deletion highest priority element will be deleted.

lets say in our priority Queue we have larger element → high pri



(-) Insertion → O(1) insert on next ava pos

(-) Deletion → O(n) ≈ O(2n)
Search for highest priority → O(n)
after removing 10 (20) → O(n)
copy all elements ahead of (20) one step back

(-) We could store elements in sorted order but that would have
Insertion → O(n)
Deletion → O(1)

(-) one operation is always costly.

(c) Max Heap / Heap data structure can be used to implement priority Queue.

Insertion -> O(log n) } -> O(log n)
Delete -> O(log n)

(c) O(log n) is smaller than O(n)

— x — x — x — x —